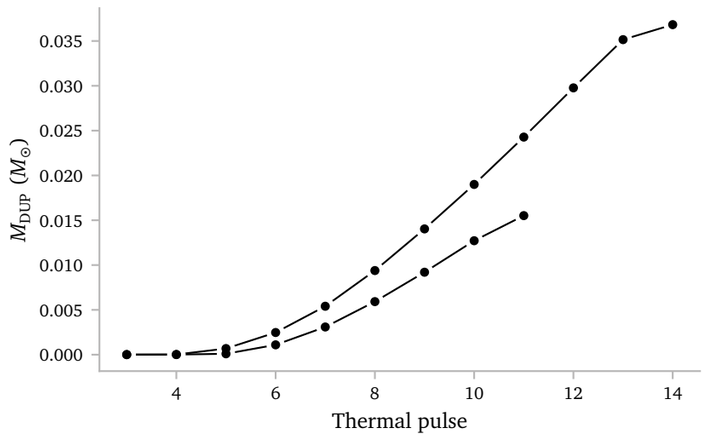
Koen Essers • Master Internship • July 29, 2026

# Notes Week 22

## a. Dredge-Up mass

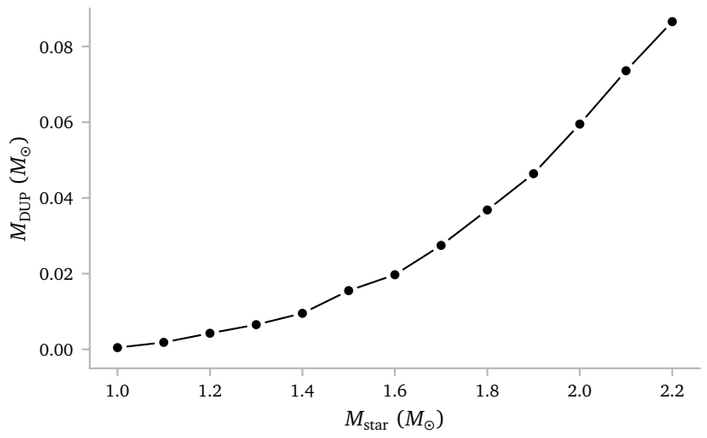
Since the current plan is to use the amount of mass that has been dredged up and the intershell abundances to infer the envelope abundances, I must be sure that I can actually accurately get this dredged up mass.

Below I show that this is indeed possible, and the values align roughly with what I would expect from Figure 1 of Karakas & Lugaro (2016).




The figure shows the amount of dredged up mass for a 1.5 $M_\odot$ and a 1.8 $M_\odot$ star with a metallicity of $z = 0.0557$. Some quick tests showed that this aligns very well with the increased CO-ratio as a function of TP-count.

Below, I plot the total amount of dredged-up mass as a function of initial star age.




This agrees quite well with Karakas & Lugaro (2016). I would expect a maximum to occur close to $M_{\rm star} = 3\ M_\odot$.

> **NOTE:** The figure above uses a grid of single star models that I simulated later. These models have a slightly different metallicity of $[\mathrm{Fe/H}] = -0.40$.

I am saving the amount of dredged-up mass in the single star `Star` objects, so it is quite easy to access this quantity later, also for grids.

Below, I print the implementation:

```
class StellarModel:
    ...
    def compute_m_DUP(self):

        lambda_DUP = np.asarray(self.lambda_DUP)[self.ntpagb :]
        he_core_mass = np.asarray(self.m_core)[self.ntpagb :]
        TP_count = np.asarray(self.TP_count)

        M_DUP = []
        TP_numbers = []

        unique_TPs = np.unique(TP_count)

        for tp in unique_TPs[2:]:  # skip before second pulse

            # indices during this thermal pulse
            pulse_idx = np.where(TP_count == tp)[0]

            if len(pulse_idx) == 0:
                continue

            # maximum lambda during pulse
            lambda_max = np.nanmax(lambda_DUP[pulse_idx])

            previous_tp = tp - 1
            previous_idx = np.where(TP_count == previous_tp)[0]

            if len(previous_idx) == 0:
                continue

            core_previous = np.min(he_core_mass[previous_idx])
            core_current = he_core_mass[pulse_idx[0]]
            delta_core = core_current - core_previous

            if delta_core <= 0:
                continue

            M_DUP.append(lambda_max * delta_core)
            TP_numbers.append(tp)

        return np.array(M_DUP)
    ...
```

It is pretty straightforward and uses the following equation,

$$\Delta M_{\rm DUP} = \lambda \cdot \Delta M_{\rm H},$$

where $\lambda$ is the dredge up efficiency parameter and $\Delta M_{\rm H}$ is the increase in core mass during the preceding interpulse. The additional routines from Rees et al. (2024) give us a good definition of $\lambda$ which we can use and the implementation of `compute_m_DUP()` is mostly determining $\Delta M_{\rm H}$.

The interpulse core mass growth is also computed by the subroutines from Rees et al. (2024), *but* I have not saved these in the history files, which makes it impossible to use these values for earlier models. I think this solution is sufficient.

## b. Envelope Abundances

As I have shown before the summer break, the surface abundance is not always equal to the envelope abundance, mainly because the surface layers are not[1] mixed efficiently.

I have implemented some additional routines my *MESA* simulations to compute the average envelope abundance of all computed isotopes. The implementation "borrows" from the implementation of the surface abundance, but using the much larger envelope region to average over.

Below I print my implementation.

1 at least according to *MESA*,

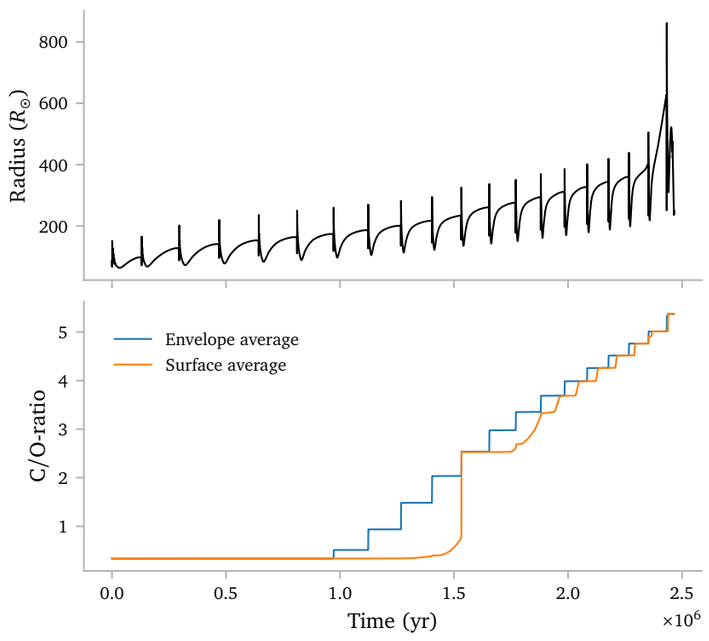
```
real(dp) function envelope_avg_x(s,j)
    use chem_def, only: chem_isos
    type (star_info), pointer :: s
    integer, intent(in) :: j
    real(dp) :: sum_x, sum_dq
    integer :: k
    include 'formats'
    if (j == 0) then
        envelope_avg_x = 0d0
        return
    end if
    sum_x = 0
    sum_dq = 0
    do k = 1, s% nz
        sum_x = sum_x + s% xa(j,k)*s% dq(k)
        sum_dq = sum_dq + s% dq(k)
        if (sum_dq >= s%xtra(x_q_above)) exit
    end do
    envelope_avg_x = sum_x/sum_dq
end function envelope_avg_x
```

It takes the star object `s`, and an isotope index `j` as input.

It uses the extra variable `s%xtra(x_q_above)` to determine when the end of the envelope has been reached. Keep in mind that *MESA* starts from the surface and walks inwards. `s%xtra(x_q_above)` gets saved during `compute_M_conv`.

> **NOTE:** I accidentally forgot to update `s%xtra(x_q_above)` in `compute_M_conv` for the binary stars. This means grids that I created during week 22 *do not* have the correct surface abundances. The single stars *do*!

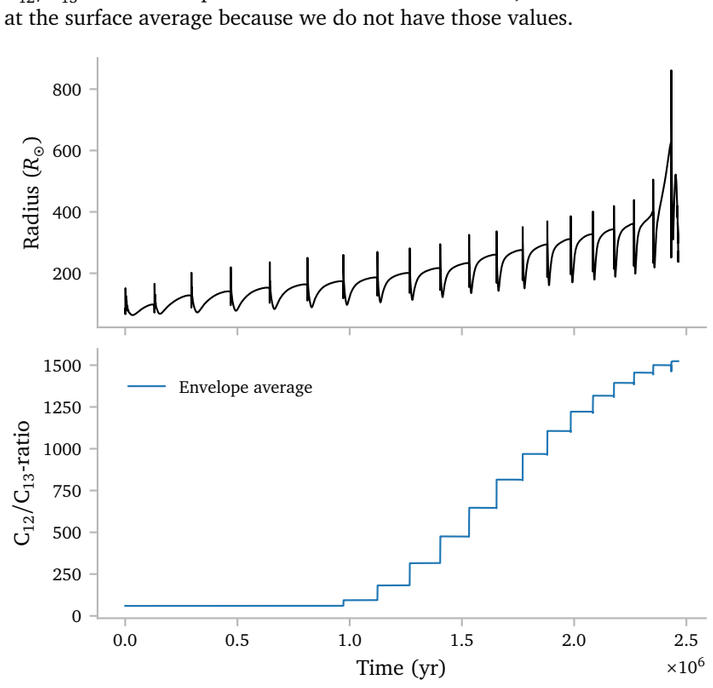
Below, I plot the envelope averaged CO-ratio and the surface average for a $z = 0.00453$, $M = 2.0\ M_\odot$ star.




This plot clearly shows the difference between the two. Although the surface average tends to approach the envelope average, they are definitely not always equal. The envelope average follows a much more predictable staircase-like pattern, which is more in line with what we expect. I think the function is behaving as it should.

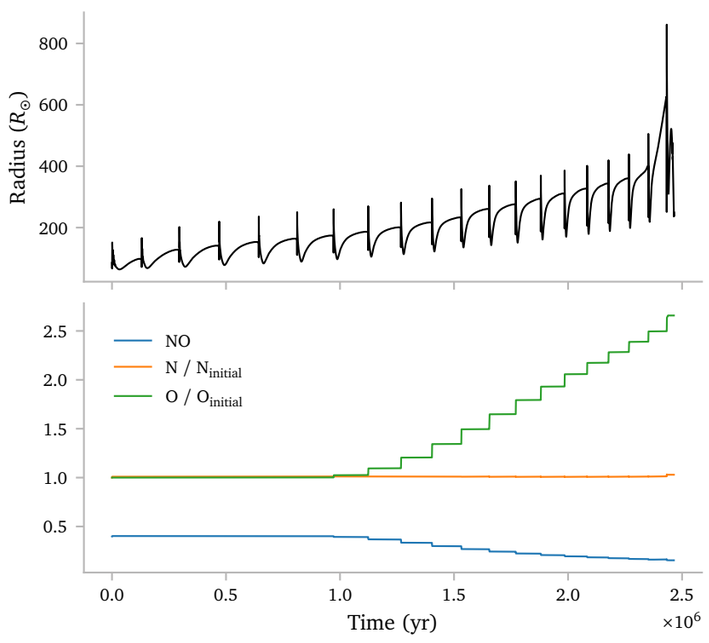
The Karakas & Lugaro (2016) models predict a CO-ratio of about 2 for a 2 $M_\odot$ star with a slightly higher metallicity of $z = 0.007$. Clearly my *MESA* models predict a much larger ratio.

Since I am keeping track of *all* of the envelope abundances, I can also plot some other ratios. Karakas & Lugaro (2016) Also show a figure for the $C_{12}/C_{13}$-ratio. I can plot this ratio as well for the star, but we cannot look at the surface average because we do not have those values.




Apparently, this ratio does not increase with the relative steps as the CO-ratio. My *MESA* model seems to overpredict this value by a factor 10 compared to the Karakas & Lugaro (2016) models. I do think this ratio is very much metallicity dependent though, comparing plots 5, 6 and 7 of this paper.

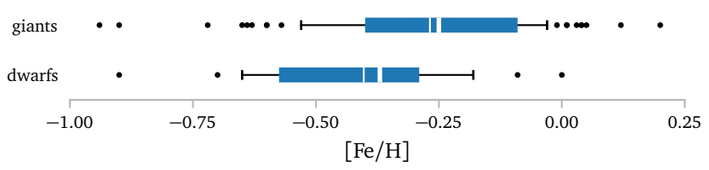
As a sanity check, we can look at the NO-ratio, which we would expect it not to rise, since there is no HBB.




We do indeed see that it does not rise. In fact, it slightly decreases as some oxygen is dredged up to the envelope.

## c. Dwarf Metallicity

Like we discussed before the summer break, we want to simulate Barium dwarfs since seem to have a smaller spread in mass and metallicity. Below I plot the metallicity distribution of the dwarfs again:




Maybe the sample from de Castro et al. (2016) has slightly more metal-rich stars, but the overall, the distributions look similar.

We decided on using $[Fe/H] = -0.4$ which equates to $z \approx 0.00557$.

From now on, all models will use this metallicity value.

## d. Extended Grid

### d.1 Idea

I want to simulate TPAGB stars from 1 to 3.5 $M_\odot$ in order to make sure I get the minimum mass for TPAGB stars.

Initially, I use a dense spacing of $\Delta M = 0.1 M_\odot$. This is probably a bit much to construct grids with, but at least I can clearly find the starting mass of the TPAGB, and I can see the stability of a wide range of masses.

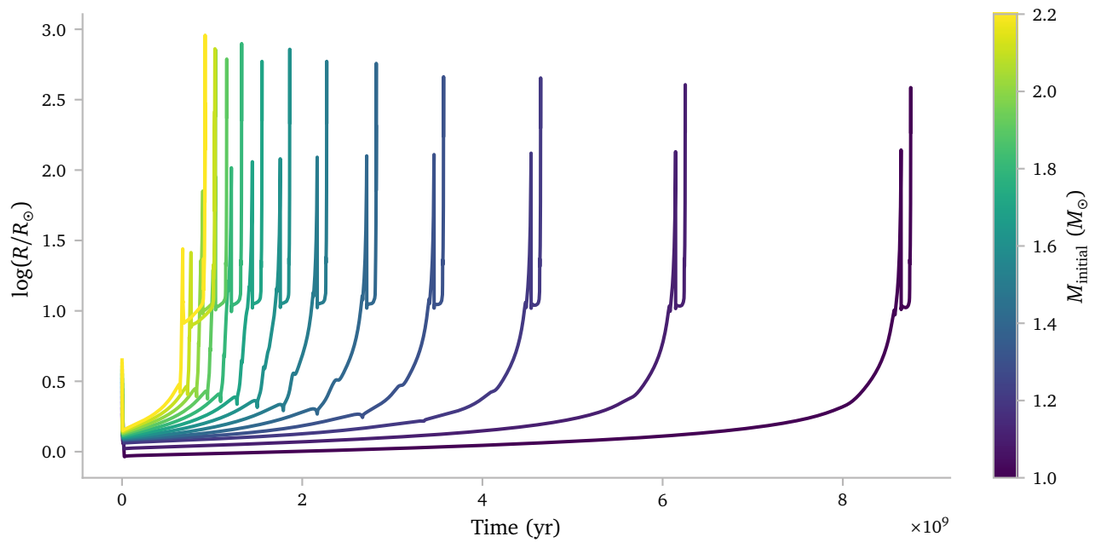
## e. Results

It turns out that MESA does have quite a bit of trouble simulating single star TPAGB stars beyond something like 2 $M_\odot$. Especially the last pulses seem to cause a lot of trouble with convergence.

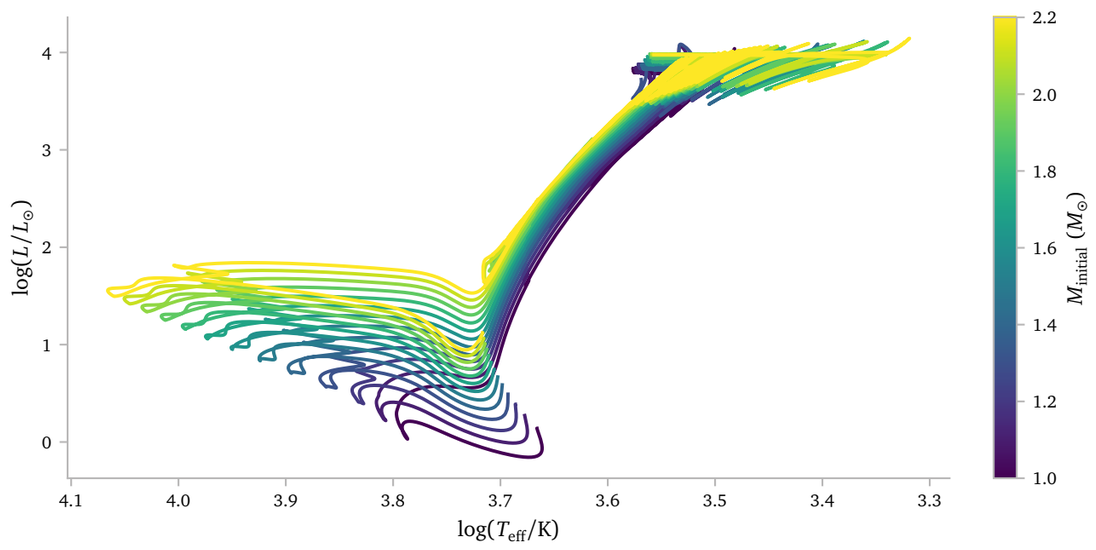
Since these stars require *a lot* of manual resuscitations to get them going again, I decided to stop at $M = 2.2\ M_\odot$, although I could definitely try to go to higher masses if this turns out to be necessary[2] .

2 I think it will not be necessary though as we will see later.

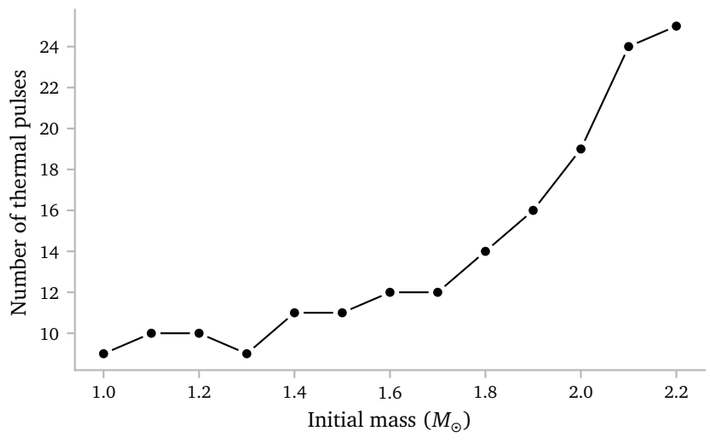
Below I plot the models of the stars from $M = 1\ M_\odot$ to $M = 2.2\ M_\odot$.



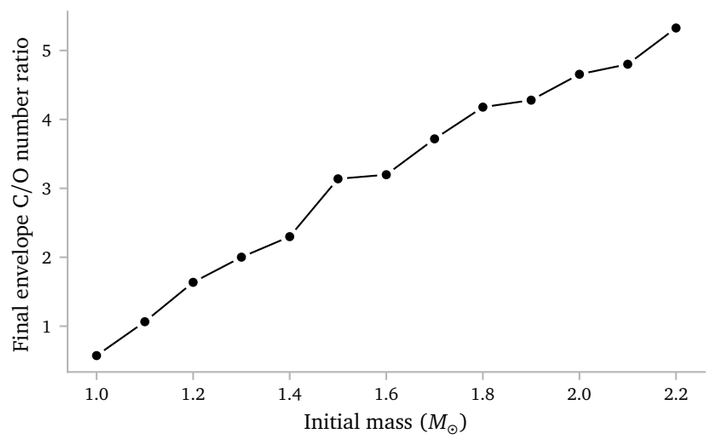

This looks good. I expect the lifetime of heavier stars to be shorter and that is exactly what i observe. We also see that the heavier stars reach larger radii, which is also expected.



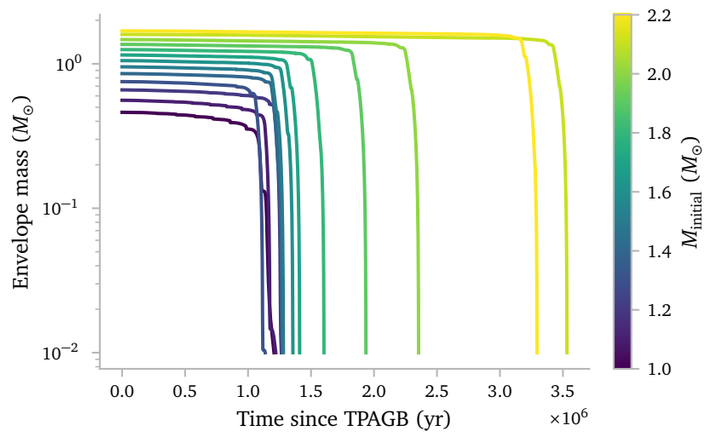

The HR also looks like what I would expect. The heavier stars are more towards the blue and luminous side on the MS. On the Giant branches, the stars are on the Hayashi tracks, where the approximate temperature is determined by the mass of the stars. Again, the results are as I would expect.




We see that the number of thermal pulses increases with mass. This is in line with the results from various studies of TPAGB-stars (eg. Rees et al., 2024; Karakas et al., 2002; Karakas & Lugaro, 2016).




The CO-ratio also increases with mass, which makes sense, since we saw that the amount of dredged up mass increases with mass too. We can almost directly compare these results with the results from Karakas & Lugaro (2016). They use a metallicity of 0.007, so slightly higher, and they find very similar C/O-ratios.




Here I show that all of the stars terminate with only 0.01 $M_\odot$ of envelope left.

The single star results are not that interesting or surprising, but they were not meant to be. I think these plots show that the stars correctly evolved throughout their evolution and can be used as input for various binary grids.

## f. Optimizing Grid Generation Code

### f.1 Idea

Since we will be dealing with much larger grids and more masses, I think it is very much time to optimize the grid generation code. Up to now, it would read in the compute star model to use as input. This is very slow.

My idea is to create some optimized binary file that contains only the necessary data for the grid generation.

I also want to be able to generate more general grids, where we can vary any parameter, not just the initial Roche lobe radius and mass ratio but also for example the initial mass or the amount of accretion.

I also want to generate some `settings.json` files so I can easily inspect which which settings the grid was generated, as well as what the settings of individual models are.

The code I used previously also could drastically use a refactor

### f.2 Implementation

The code now neatly starts with the user variables:

```
# WARNING: CHECK / MODIFY THESE PATHS
grid_name = "epsilon-grid-test"
proj_dir = "/home/koen/master-internship"
reference_binary_dir =
 ↪  f"{proj_dir}/mesa-models/reference-binary/2026-07-01/"
binary_exe_dir =
 ↪  f"{proj_dir}/mesa-models/reference-binary/"

# WARNING: SETTINGS FOR THE GRID
single_star_dirs =
 ↪  [f"{proj_dir}/mesa-models/single-stars/z0.00557/completed/M1.5"]
Rs = [350]
qs = np.linspace(0.4, 1, 7)
epss = np.linspace(0, 0.85, 3)

mass_transfer_beta = np.linspace(0, 1, 7)
mass_transfer_delta = 1 - mass_transfer_beta

# WARNING: SETTINGS FOR BINARY SIMULATION
mass_transfer_alpha = 0.0
mass_transfer_gamma = 1.44
```

I then do some general setup things like creating the folder, setting some paths and getting the metallicity.

```
grid_dir = f"{proj_dir}/mesa-models/{grid_name}/"
models_dir = f"{single_star_dirs[0]}/models/"
os.makedirs(os.path.dirname(grid_dir), exist_ok=True)
shutil.copyfile(f"{binary_exe_dir}/binary",
 ↪  f"{grid_dir}/binary")


def get_initial_Z():
    model = [os.path.basename(f) for f in
     ↪  os.scandir(models_dir)][0]
    Z = mr.MesaData(f"{models_dir}/{model}").initial_z
    return Z


star_Z = get_initial_Z()
```

For the rest of process, I use two helper classes:

```
class Evolution:
    def __init__(self, bin, Star, inds):
        self.bin = bin
        self.star = Star
        self.inds = inds

class GridPoint:
    def __init__(self, evolution, model):
        self.evolution = evolution
        self.model = model
```

I then have the following functions to create the setting files:

```
def generate_settings_file():
def generate_run_file(
    R, q, model_name, age, m1, m2, a, omega, varcontrol,
     ↪  beta, delta, eps, run_dir
):
```

I call `generate_settings_file()` immediately and use the `generate_run_file()` only during the grid generation process.

The grid generation loop looks like this:

```
for single_star_dir in single_star_dirs:
    for R in Rs:
        for q in qs:
            print(R, q)
            base = {
                "R": R,
                "q": q,
                "single_star_dir": single_star_dir,
            }

            grid_point = prepare_grid_point(params=base)

            if grid_point is None:
                print("no suitable model")
                continue
            if grid_point is False:
                print("skipping this R")
                break

            for eps in epss:
                if eps == 1:
                    params = {
                        **base,
                        "beta": 0,
                        "delta": 0,
                        "eps": eps,
                    }

                    generate_run(params=params,
                     ↪  grid_point=grid_point)
                    continue

                for beta, delta in
                 ↪  zip(mass_transfer_beta,
                 ↪  mass_transfer_delta):
                    params = {
                        **base,
                        "beta": beta * (1 - eps),
                        "delta": delta * (1 - eps),
                        "eps": eps,
                    }

                    generate_run(params=params,
                     ↪  grid_point=grid_point)
```

These are the three variables that change the simple integration outcome from the `evolve.py` script that I am using. I split these from the other variables to prevent unnecessary double computation.

Here, I do the bulk of the computation, including the simple integration.

Next there is some logic to stop the generation of the current $R$ if a model failed already

Finally, I add the other variables that do not change the computation, only the MESA settings and I change all of the necessary files.

This all happens here.

For completeness, This is what `prepare_grid_point()` looks like:

```
def prepare_grid_point(params):
    evolution = evolve_binary(params)
    if evolution is None:
        return None

    contact_age = find_contact(evolution)
    if contact_age is None:
        return None

    if contact_age is False:
        return False

    models_dict = get_model_dict(params)
    model = select_model(contact_age, models_dict)
    if model is None:
        return None

    return GridPoint(evolution, model)
```

and this is what `generate_run()` looks like:

```
def generate_run(params, grid_point):

    run_dir = prepare_run_dir(params, grid_point.model)

    write_run_files(
        params,
        grid_point.model,
        grid_point.evolution,
        run_dir,
    )
```

I tested this new setup with a general grid that I explore in section i.

## f.3 New MesaGrid class

Since the new grids are a lot more complicated, the old `MesaGrid` class that I was could not understand the structure of the new class and I needed something more general.

The new class is still a bit barren and I am planning to expand it based on what I need. For now it has the following member functions and data:

```
class MesaGrid:
    def __init__(self, grid_dir):

        self.grid_dir = Path(grid_dir)

        with open(f"{grid_dir }/grid_settings.json") as
         ↪  f:
            self.settings = json.load(f)

        self.axes = {
            "R": self.settings["Rs"],
            "q": self.settings["qs"],
            "beta":
             ↪  self.settings["mass_transfer"]["beta"],
            "delta":
             ↪  self.settings["mass_transfer"]["delta"],
            "eps": self.settings["eps"],
        }

        self._load_models()

    def _load_models(self):
        self.models = []
        ...
```

```
def filter(self, **filters):

    for run in self.models:

        keep = True

        for key, values in filters.items():

            if not isinstance(values, (list, tuple,
             ↪  set)):
                values = {values}

            if run.params[key] not in values:
                keep = False
                break

        if keep:
            yield run

def array(self, value, x, y, **filters):

    xs = self.axes[x]
    ys = self.axes[y]

    arr = np.full((len(xs), len(ys)), np.nan)

    x_index = {np.round(v, 3): i for i, v in
     ↪  enumerate(xs)}
    print(x_index)
    y_index = {np.round(v, 3): i for i, v in
     ↪  enumerate(ys)}

    for run in self.filter(**filters):

        if callable(value):
            z = value(run)
        else:
            z = getattr(run, value)
```

```
            if x in ["beta", "delta"]:
                i = x_index[np.round(run.params[x] / (1 -
                 ↪  run.params["eps"]), 3)]
            else:
                i = x_index[np.round(run.params[x], 3)]

            if y in ["beta", "delta"]:
                j = y_index[np.round(run.params[y] / (1 -
                 ↪  run.params["eps"]), 3)]
            else:
                j = y_index[np.round(run.params[y], 3)]

            arr[i, j] = z

        return np.asarray(xs), np.asarray(ys), arr


class MesaRun:

    def __init__(self, run_dir):
        self.run_dir = run_dir

        with open(run_dir / "settings.json") as f:
            self.params = json.load(f)

        self.get_history()

        self.env_mass = self.history.envelope_mass
        self.q = self.history.star_2_mass /
         ↪  self.history.star_1_mass
        self.age = self.params["starting_age"] +
         ↪  self.star_age

    def __getattr__(self, name):
        return getattr(self.history, name)

    def get_history(self):
        try:
            with open(f"{self.run_dir}/history.pkl",
             ↪  "rb") as f:
                self.history = pickle.load(f)
        except:
            print(f"reading
             ↪  {self.run_dir}/LOGS/TPAGB/history.data")
            self.history =
             ↪  mr.MesaData(f"{self.run_dir}/LOGS/TPAGB/history.data")
            with open(f"{self.run_dir}/history.pkl",
             ↪  "wb") as f:
                pickle.dump(self.history, f,
                 ↪  protocol=pickle.HIGHEST_PROTOCOL)

    def get_profiles(self):

        try:
            with open(f"{self.run_dir}/profiles.pkl",
             ↪  "rb") as f:
                self.profiles = pickle.load(f)

        except:
            profiles_dict =
             ↪  mr.MesaLogDir(f"{self.run_dir}/LOGS/TPAGB/")
            for i, profile in
             ↪  enumerate(profiles_dict.profile_numbers):
                _ =
                 ↪  profiles_dict.profile_data(profile_number=profile)
            self.profiles =
             ↪  list(profiles_dict.profile_dict.values())
            with open(f"{self.run_dir}/profiles.pkl",
             ↪  "wb") as f:
                pickle.dump(self.profiles, f,
                 ↪  protocol=pickle.HIGHEST_PROTOCOL)
```

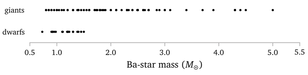
Here, I also added a `MesaRun` class to neatly combine the mesa history data as well as the parameters of the run.

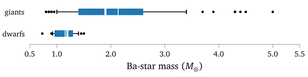
## g. Debugging!

The 2.4 $M_\odot$ star was behaving *REALLY* badly, and would not at all continue its evolution, despite all sorts of attempts to help the solver.

I tried to debug this star and after looking into a bunch of things, I was ultimately unsuccessful in resolving the issue. My full documentation can be found HERE: B

## h. Ba-dwarf masses

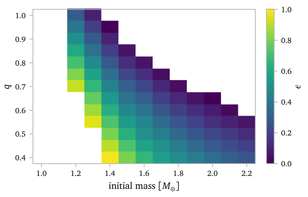


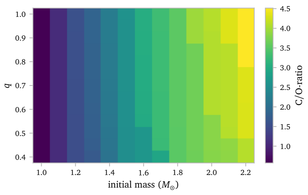



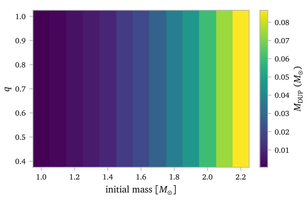

When considering the Barium star masses, the dwarfs are much more easily modelled by a single value around $M_{\rm Ba} = 1.2\ M_\odot$. This aligns well with the results from Sarmah et al. (2026), who find an average mass of $M_{\rm Ba} = 1.29 \pm 0.09\ M_\odot$.

From now on, I will use this value.

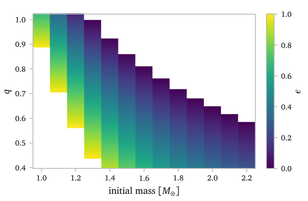
Let's say the binary interacts at a Roche lobe radius of 450 $M_\odot$, we find:

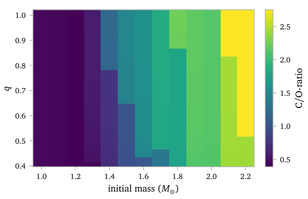


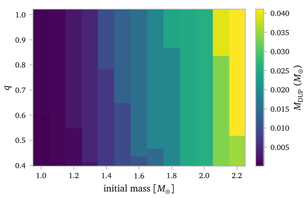




Now, for a Roche lobe radius of 450 $M_\odot$, we find:



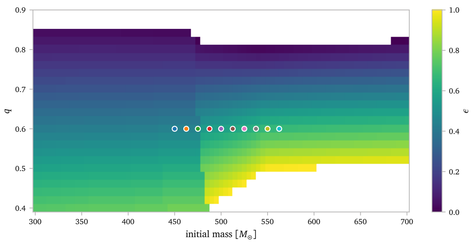

It is clear that the heavy stars have trouble producing the barium star masses.

Now, obviously, there might be positive correlation between the barium star mass and the TP-AGB mass, but it should still be obvious that only the lighter TP-AGB stars can accrete significant mass without starting with a very unequal mass ratio.

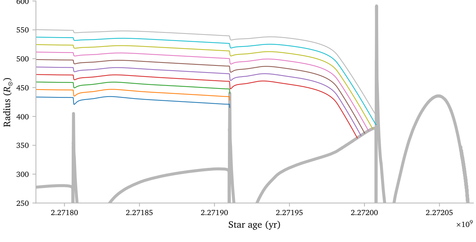
### h.1 Changing interaction radius

Instead of varying the mass, we can also vary the interaction radius. This setup is lot more like my previous grids.

For the $M_i = 1.5\ M_\odot$ model, we get the following plot:



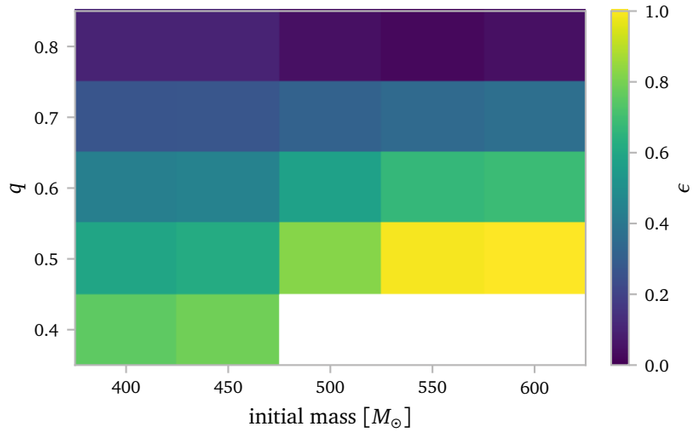

There are two visible boundaries here. The first one is very obvious and occurs at the point where the Vassiliadis & Wood (1993) winds start. The second boundary is a bit less obvious. It occurs when the models interact during the last pulse instead of during the interpulse before the last pulse. The figure below shows this for a constant $q$.




The colors of the lines correspond to the colors of the dots of the previous figure. It now also makes sense that the possible $q$-range shrinks for larger $R$. Clearly, here the star has already lost a lot of mass due to winds, which it can no longer accrete during RLOF. The winds also transfer some mass, which shifts the upper limit down a little bit[3].

3 although a lot less than the lower limit shifts upwards.

We can use something like this to generate a grid as well:




My idea is to use the $\varepsilon$ value to ensure we produce a barium star with the appropriate mass. Once we have a value for $\varepsilon$, we can then vary $\beta$ and $\delta$ to produce the required $\varepsilon$ value.

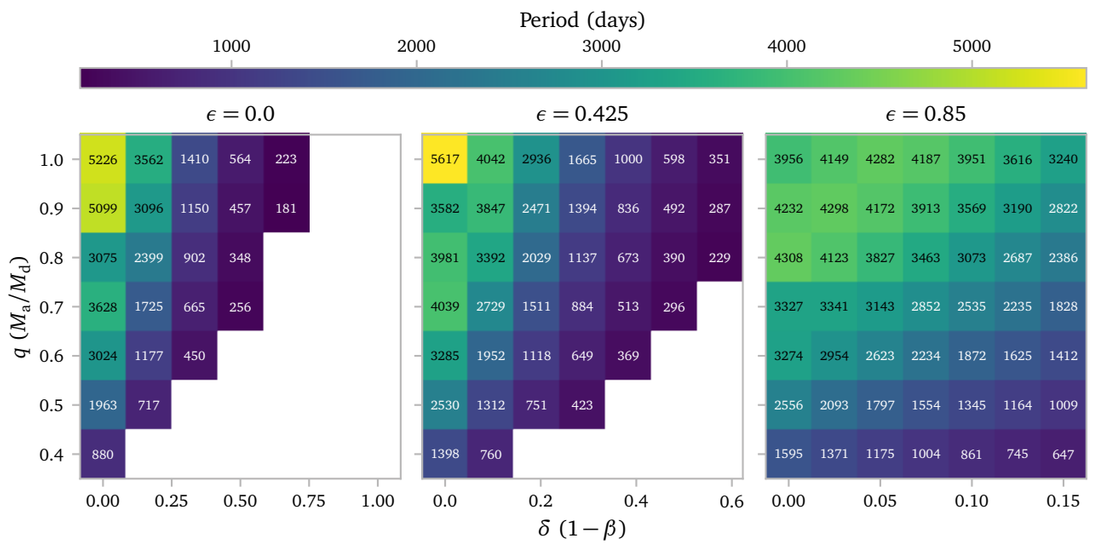
With 5 different ratios between $\delta$ and $\beta$, this gives $22 \times 5 = 110$ models. Because one of the models is so close to $\varepsilon = 0$, we actually only need to simulate 106 models, which is pretty doable in a couple of days.

Currently the grid is simulating on the Coma cluster.

# i. Grid

## i.1 History

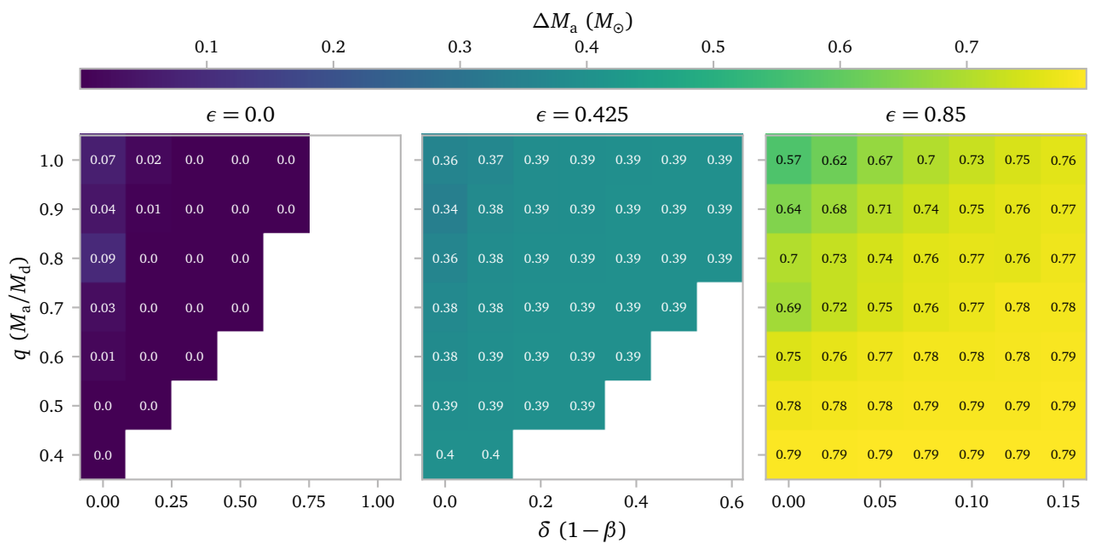
My grid has $R_{\rm RL} = 350\ R_\odot$ and three different epsilon values. For every epsilon value, I vary the initial mass ratio and the amount of mass that gets lost through isotropic re-emission and a CCT.




In the figure above, I plot the orbital period at the end of the evolution. Clearly, we are able to produce some pretty short periods, even when we have a very high accretion efficiency[4]. For the figures with a lot of accretion losses, we see that a lot of models fail. I think this is due to the Darwin instability. We do see very short periods close to the Darwin instability region.

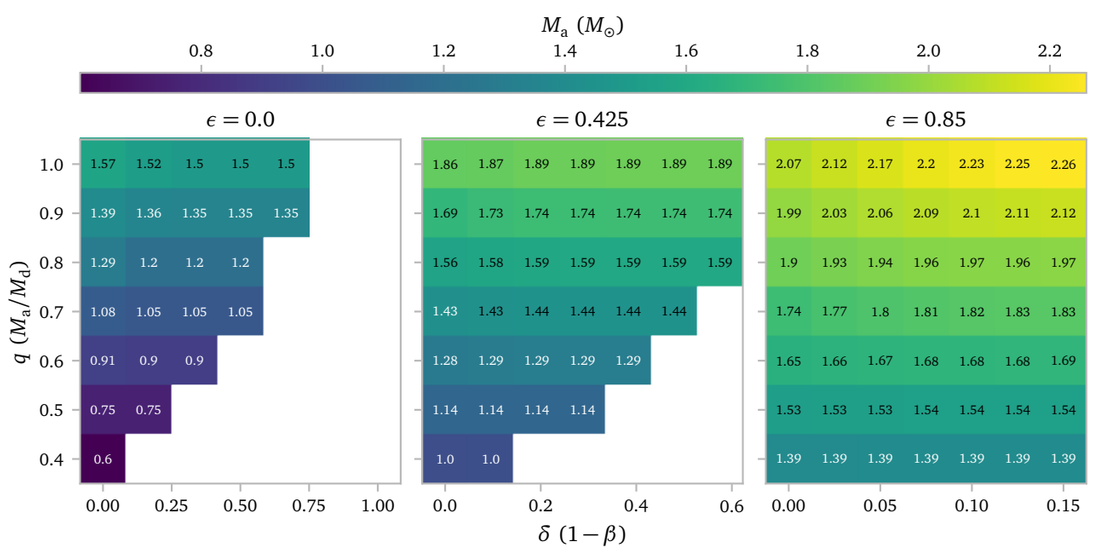
[4] i.e. the $\varepsilon = 0.85$ figure.

I think the overall trends in the figures and between the figures make sense. We see that an increased $\delta$ causes shorter periods with $q$ having an effect on the strength of the shrinkage as well. We also see that less accretion losses cause less shrinkage, which is what we expect as well.

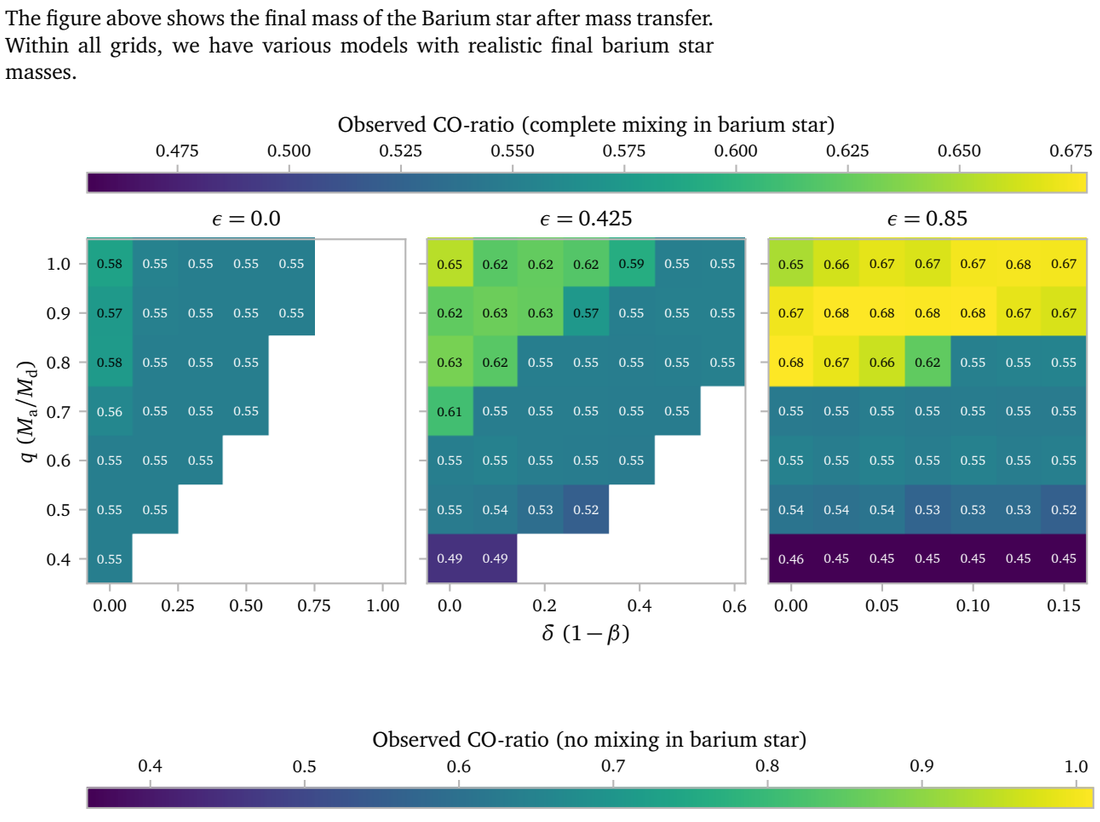



The plot above shows the amount of accreted mass. Now, it makes sense that it is very close to zero for the $\varepsilon = 0$ grid, since obviously here all transferred mass is lost through isotropic re-emission and CCT mass loss.

The other two plots show approximately $(1.5-0.6)\times 0.425 \approx 0.38$ and $(1.5-0.6)\times 0.85 \approx 0.765$. There are some models close to $\delta = 0$ and $q = 1$ that have more wind, which decreases the amount of transferred mass.




The figure above shows the final mass of the Barium star after mass transfer. Within all grids, we have various models with realistic final barium star masses.




> **NOTE:** The figures above use the *surface* abundances because the binary models do not compute the envelope abundances correctly for this grid. I expect that the differences are minor however.

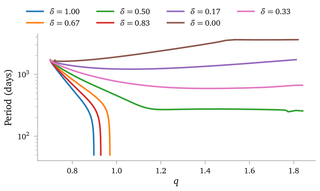
The two figures above show the CO-ratio of the barium star after mass transfer for two different mixing assumptions. The first figure shows the CO-ratio if the accreted material is completely mixed throughout the whole star.

The second figure shows the CO-ratio if the accreted material is not mixed at all, and thus lands at the surface and stays there.

The CO-ratio of the $\varepsilon = 0$ grid still changes due to the wind accretion.

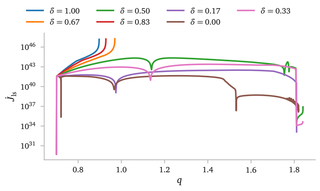
Since these models interact before the TPAGB-star has dredged up a lot of carbon, we do not produce a very carbon rich barium star. To me, it is uncertain if the star would have a detectable barium (over-)abundance.

Let's look a bit closer at the *failing models*. Below I plot the period evolution as a function of $q$ for the models starting with $q = 0.7$ and $\varepsilon = 0$.




We see the inspiral that we saw for some models in previous grids as well. I think this is due to the Darwin Instability.

To ensure this is the cause of the inspiral, we can look at the angular momentum derivative of the spin orbit coupling.

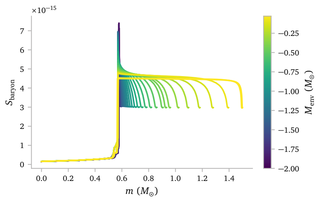



Note that the label says $\dot{J}_{\rm ib}$ but it should be $|\dot{J}_{\rm ib}|$. The derivative is negative at the time of the inspiral.

We see the strong ramp-up in spin-orbit coupling, which makes me believe it is truely the Darwin instability.

## i.2 Profiles

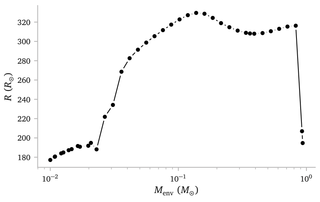
This section shows some tests for the MESA profiles. To start, here we can clearly see the profile saving code working.




The binary saves at most 50 profiles up to $M_{\rm env} = 0.001\ M_\odot$. The profiles are saved with a logarithmic spacing between the initial envelope mass and the final envelope mass. Most models will terminate before reaching $M_{\rm env} = 0.001\ M_\odot$ and will therefore have slightly fewer profiles. In this case, we have 38 models going down to $M_{\rm env} = 0.01\ M_\odot$.

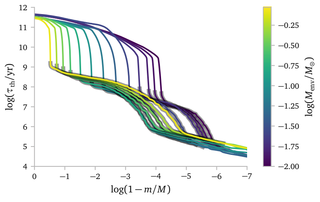
Here we can cleary see that the entropy profile of the star becomes a lot less flat once it has shed a lot of its envelope.

As a second test, I plot the radius of the profiles as a function of the envelope mass.

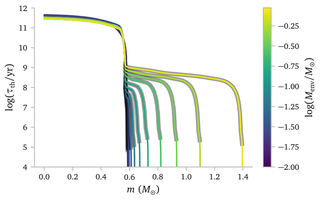



This plot not only shows the equal spacing of the profiles in log space, but also how there is not a simple relation between the envelope mass and the radius of the star. This is in part obviously caused by the thermal pulses.

Then, I plot the thermal time to the surface as a function of internal mass.




This plot aligns well with the results from Temmink et al. (2023). There, we see that an increased radius tends to shift the superadiabatic layers further from the surface. We see this here as well, with the $\log(M_{\rm env}) \approx -1$ models having the deepest superadiabatic layers.




Here, we can quite clearly see that most of the envelope op the TPAGB stars is completely superadiabatic.

## References


de Castro, D. B., Pereira, C. B., Roig, F., et al. (2016) *Chemical abundances and kinematics of barium stars*. MNRAS459, 4299.

Karakas, A. I., Lattanzio, J. C., & Pols, O. R. (2002) *Parameterising the Third Dredge-up in Asymptotic Giant Branch Stars*. PASA19, 515.

Karakas, A. I. & Lugaro, M. (2016) *Stellar Yields from Metal-rich Asymptotic Giant Branch Models*. ApJ825, 26.

Paxton, B., Bildsten, L., Dotter, A., et al. (2011) *Modules for Experiments in Stellar Astrophysics (MESA)*. ApJS192, 3.

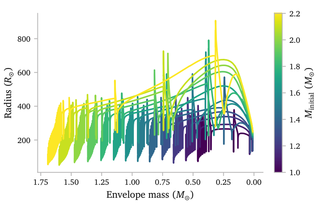
Rees, N. R., Izzard, R. G., & Karakas, A. I. (2024) *Thermal pulses with MESA: resolving the third dredge-up*. MNRAS527, 9643.

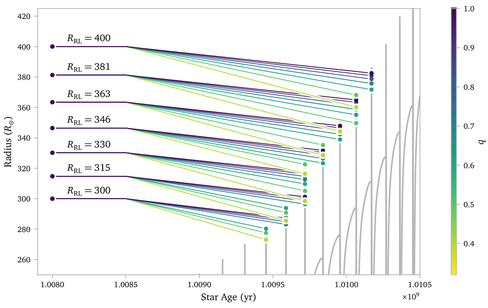
Sarmah, L., Bharat Kumar, Y., Campbell, S. W., Maben, S., & Reddy, B. E. (2026) *Unveiling the nature of barium stars. I. Asteroseismic masses and the evolutionary link between Ba dwarfs and giants*. arXiv e-prints, arXiv:2606.25517.

Temmink, K. D., Pols, O. R., Justham, S., Istrate, A. G., & Toonen, S. (2023) *Coping with loss. Stability of mass transfer from post-main-sequence donor stars*. A&A669, A45.

Vassiliadis, E. & Wood, P R. (1993) *Evolution of Low- and Intermediate-Mass Stars to the End of the Asymptotic Giant Branch with Mass Loss*. ApJ413, 641.


## A Additional Stuff

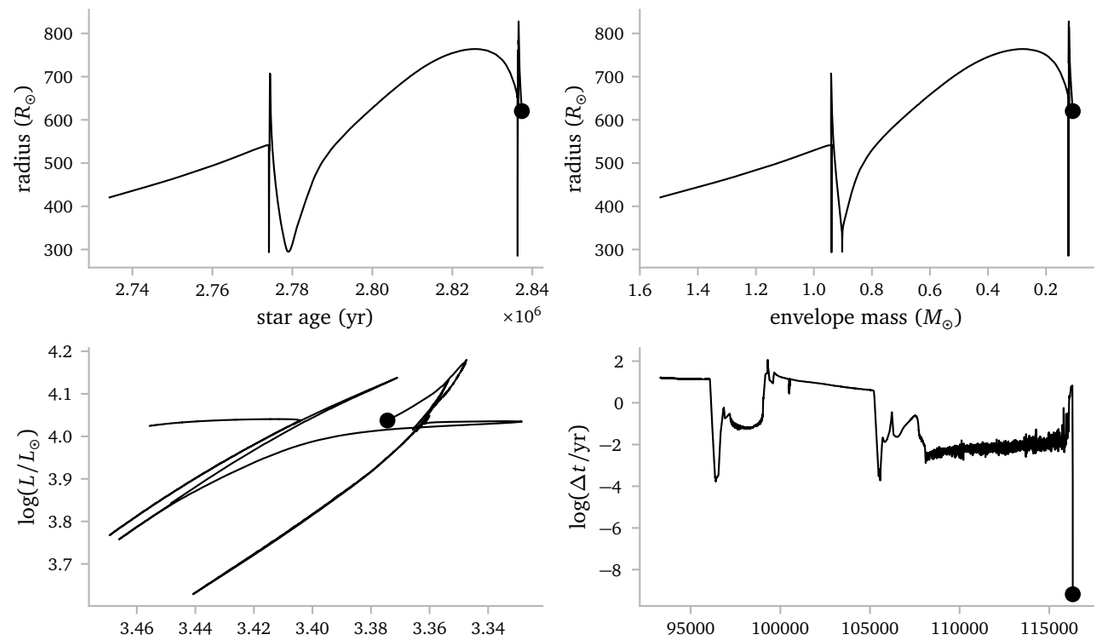


## B Debugging

The 2.4 $M_\odot$ star was behaving *REALLY* badly, and would not at all continue its evolution, despite all sorts of attempts to help the solver.

Because of this, I decided to look into *what* exactly is going wrong. Let me start by plotting where the problem starts occuring.




It seems like the star cannot continue evolving just after its last thermal pulse. The radius and HR diagram look fine, but we see that the timestep goes to 0.

The terminal output shows me the usual ÆSOPUS warning, as well as the eos warning. I need to dig a bit deeper to figure out what is actually going wrong.

To start, I created a model just before the point where the star crashes. I also use the general debug options available to me in the inlists.

To start, I see this:

```
               solver_call_number           1
             gold tolerances level          1
   correction tolerances: norm, max    3.0000000000000001D-05    3.0000000000000001D-03
tol1 residual tolerances: norm, max    9.9999999999999994D-12    1.0000000000000001D-09
1  1  coeff 1.0000 avg resid  0.691E-05     max resid dv_dt     1  0.83380E+00 mix type xx000
                  avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
                  avg+max corr+resid
1  2  coeff 1.0000 avg resid  0.683E-05     max resid dv_dt     1  0.82426E+00 mix type xx000
                  avg corr  0.573E-04      max corr lnd     2  0.12612E-01 mix type x0000
                  avg+max corr+resid
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos_for_cell: get_eos ierr          -1
do_eos returned ierr          -1
set_hydro_vars failed in call to set_micro_vars
                  set_hydro_vars returned ierr          -1
  eval_equations: set_solver_vars returned ierr          -1
adjust_correction: eval_equations returned ierr          -1    1.0D+00    1.0D+00
```

The first error occurs in the `do_eos_for_cell` call!

I then use the GNU Project Debugger `gdb` to debug *MESA*. I write:

```
> OMP_NUM_THREADS=1 gdb ./star
(gdb) b eos_support.f90:89
(gdb) condition 1 logRho .lt. -25
```

The first line activates the debugger with the `star` executable where `OMP_NUM_THREADS=1` makes sure we are not multithreading. The second line creates a breakpoint at line 89 from the file `eos_support.f90`. Line 89 is the condition in `get_eos` that returns the error -1. The third line tell the breakpoint to only break if `logRho` < -25, which is the exact condition that returns the error.

This is the (relevant) output.

```
               solver_call_number           1
             gold tolerances level          1
   correction tolerances: norm, max    3.0000000000000001D-05    3.0000000000000001D-03
tol1 residual tolerances: norm, max    9.9999999999999994D-12    1.0000000000000001D-09
1  1  coeff 1.0000 avg resid  0.691E-05     max resid dv_dt     1  0.83380E+00 mix type xx000
                  avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
                  avg+max corr+resid
1  2  coeff 1.0000 avg resid  0.683E-05     max resid dv_dt     1  0.82426E+00 mix type xx000
                  avg corr  0.573E-04      max corr lnd     2  0.12612E-01 mix type x0000
                  avg+max corr+resid

Breakpoint 1, eos_support::get_eos (s=0xe7c0e0 <__star_data_def_MOD_star_handles>, k=1,
                                    xa=..., rho=<optimized out>, logrho=-40.561141182088775,
                                    t=228.31745818308877, logt=2.3585391208756734,
                                    res=..., dres_dlnrho=..., dres_dlnt=..., dres_dxa=...,
                                    ierr=0) at ../private/eos_support.f90:89
89            if(logRho < -25) then
```

This is the relevant source code:

```
59  ! $MESA_DIR/star/private/eos_support.f90
60
61  subroutine get_eos( &
62             s, k, xa, &
63             Rho, logRho, T, logT, &
64             res, dres_dlnRho, dres_dlnT, &
65             dres_dxa, ierr)
66
67          use eos_lib, only: eosDT_get
68          use eos_def, only: num_eos_basic_results,
            ↪  num_eos_d_dxa_results, num_helm_results,
            ↪  i_lnE
69
70          type (star_info), pointer :: s
71          integer, intent(in) :: k ! 0 means not being
            ↪  called for a particular cell
72          real(dp), intent(in) :: xa(:), Rho, logRho, T,
            ↪  logT
73          real(dp), dimension(num_eos_basic_results),
            ↪  intent(out) :: &
74             res, dres_dlnRho, dres_dlnT
75          real(dp), intent(out) ::
            ↪  dres_dxa(num_eos_d_dxa_results,s% species)
76          integer, intent(out) :: ierr
77
78          real(dp), dimension(num_eos_basic_results) ::
            ↪  dres_dabar, dres_dzbar
79          integer :: j
80          logical :: off_table
81
82          include 'formats'
83
84          ierr = 0
85
86          if (s% doing_timing) &
87              s% timing_num_get_eos_calls = s%
                ↪  timing_num_get_eos_calls + 1
88
89          if(logRho < -25) then
90              ! Provide some hard lower limit on what we
                ↪  would even try to evalue the eos at
91              ! Going to low causes FPE's when we try to
                ↪  evaluate certain derviatives that need
                ↪  (rho**power)
92              s% retry_message = 'eos evaluted at too low a
                ↪  density'
93              ierr = -1
94              return
95          end if
96      ...
```

Clearly, something is going wrong with the *density*! In this case, the first zone (k=1, at the surface) has a density of $\log(\rho) \approx -40$. Obviously this is insane. The temperature is very low as well and wrong.

So, really we need to look back a little bit, not at `get_eos` where the *error* originates, but a little before that, where the back values originate.

Below, I show the source code that calls `get_eos`:

```
! $MESA_DIR/star/private/micro.f90

subroutine do_eos_for_cell(s,k,ierr)

    use const_def
    use chem_def
    use chem_lib
    use eos_def
    use eos_lib, only: eval_PC_fraction
    use eos_support
    use net_def,only:net_general_info
    use star_utils, only: write_eos_call_info

    type (star_info), pointer :: s
    integer, intent(in) :: k
    integer, intent(out) :: ierr

    real(dp), dimension(num_eos_basic_results) :: &
         res, res_a, res_b, d_dlnd, d_dlnT, d_eos_dabar, d_eos_dzbar
    real(dp) :: &
         sumx, dx, dxh_a, dxh_b, &
         Rho, logRho, lnd, lnE, logT, T, energy, logQ, frac
    integer, pointer :: net_iso(:)
    integer :: j, species
    real(dp), parameter :: epsder = 1d-4, Z_limit = 0.5d0

    logical, parameter :: testing = .false.

    include 'formats'

    ierr = 0

    net_iso => s% net_iso
    species = s% species
    call basic_composition_info( &
         species, s% chem_id, s% xa(1:species,k), s% X(k), s% Y(k), s% Z(k), &
         s% abar(k), s% zbar(k), s% z2bar(k), s% ye(k), s% mass_correction(k), sumx)

    logT = s% lnT(k)/ln10

    if (s% lnPgas_flag) then
       if (s% Pgas(k) == 0) then
          ierr = -1
          return
          write(*,2) 's% lnPgas(k)/ln10', k, s% lnPgas(k)/ln10
          stop 'do_eos_for_cell s% Pgas(k) == 0'
       end if
       call get_peos( &
            s, k, s% Z(k), s% X(k), s% abar(k), s% zbar(k), s% xa(:,k), &
            s% Pgas(k), s% lnPgas(k)/ln10, s% T(k), logT, &
            Rho, logRho, s% dlnRho_dlnPgas_const_T(k), s% dlnRho_dlnT_const_Pgas(k), &
            res, s% d_eos_dlnd(:,k), s% d_eos_dlnT(:,k), &
            d_eos_dabar, d_eos_dzbar, ierr)
       s% lnd(k) = logRho*ln10
       s% rho(k) = Rho
       call store_stuff(ierr)
       return
    end if

    logRho = s% lnd(k)/ln10

    call get_eos( &
         s, k, s% Z(k), s% X(k), s% abar(k), s% zbar(k), s% xa(:,k), &
         s% rho(k), logRho, s% T(k), logT, &
         res, s% d_eos_dlnd(:,k), s% d_eos_dlnT(:,k), &
         d_eos_dabar, d_eos_dzbar, &
         ierr)
    if (ierr /= 0) then
       if (s% report_ierr) then
          write(*,*) 'do_eos_for_cell: get_eos ierr', ierr
          stop 'do_eos_for_cell'
       end if
       return
    end if
    s% lnPgas(k) = res(i_lnPgas)
    s% Pgas(k) = exp_cr(s% lnPgas(k))
    ...
```

The errors occur in line 344. We can also look at the backtrace of the get_eos call:

```
(gdb) bt
#0  eos_support::get_eos (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, k=1, xa=...,
 ↪  rho=<optimized out>, logrho=-40.561141182088775,
 ↪  t=228.31745818308877, logt=2.3585391208756734,
    res=..., dres_dlnrho=..., dres_dlnt=...,
     ↪  dres_dxa=..., ierr=0) at
     ↪  ../private/eos_support.f90:89
#1  0x0000000000503d11 in micro::do_eos_for_cell
 ↪  (s=<optimized out>, k=<optimized out>, ierr=0) at
 ↪  ../private/micro.f90:385
#2  0x00000000004422ec in
 ↪  star_utils::__star_utils_MOD_foreach_cell._omp_fn.0
 ↪  () at ../private/star_utils.f90:72
#3  0x00007fea9e211f42 in GOMP_parallel (fn=0x442260
 ↪  <star_utils::__star_utils_MOD_foreach_cell._omp_fn.0>,
 ↪  data=0x7ffcbdc100b0, num_threads=1, flags=0)
    at
     ↪  /home/user/sdk2-tmp/build/gcc/libgomp/parallel.c:171
#4  0x0000000000456935 in star_utils::foreach_cell
 ↪  (s=0xe7c0e0 <__star_data_def_MOD_star_handles>,
 ↪  nzlo=<optimized out>, nzhi=<optimized out>,
 ↪  use_omp=<optimized out>,
    do1=0x5036c0 <micro::do_eos_for_cell>, ierr=0) at
     ↪  ../private/star_utils.f90:72
#5  0x00000000005057c3 in micro::do_eos (ierr=0,
 ↪  nzhi=<optimized out>, nzlo=<optimized out>,
 ↪  s=<optimized out>) at ../private/micro.f90:271
#6  micro::set_eos (ierr=0) at ../private/micro.f90:217
#7  micro::set_micro_vars (nzlo=1, nzhi=5857,
 ↪  skip_eos=.FALSE., skip_net=.FALSE., skip_neu=.FALSE.,
 ↪  skip_kap=.FALSE., ierr=0) at ../private/micro.f90:97
#8  0x000000000050cbe7 in hydro_vars::set_hydro_vars
 ↪  (s=0xe7c0e0 <__star_data_def_MOD_star_handles>,
 ↪  nzlo=1, nzhi=5857, skip_basic_vars=.TRUE.,
 ↪  skip_micro_vars=.FALSE.,
    skip_m_grav_and_grav=.TRUE., skip_eos=.FALSE.,
     ↪  skip_net=.FALSE., skip_neu=.FALSE.,
     ↪  skip_kap=.FALSE., skip_grads=.TRUE.,
     ↪  skip_rotation=.TRUE., skip_brunt=.TRUE.,
    skip_other_cgrav=.TRUE., skip_mixing_info=.TRUE.,
     ↪  skip_set_cz_bdy_mass=.TRUE., skip_mlt=.FALSE.,
     ↪  ierr=0) at ../private/hydro_vars.f90:506
#9  0x00000000007fa3fd in
 ↪  solver_support::set_vars_for_solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13, nzlo=1,
 ↪  nzhi=<optimized out>, dt=69894018.156770468, ierr=0)
    at ../private/solver_support.f90:1107
#10 0x00000000007fe7a9 in solver_support::set_solver_vars
 ↪  (ierr=0, dt=69894018.156770468, nvar=13, s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>) at
 ↪  ../private/solver_support.f90:815
#11 solver_support::eval_equations (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13, ierr=0)
 ↪  at ../private/solver_support.f90:113
#12 0x00000000007bd036 in do_solver::do_equations
 ↪  (ierr=0) at ../private/star_solver.f90:736
#13 0x0000000000c2960 in star_solver::adjust_correction
 ↪  (_err_msg=256, _err_msg=256, ierr=0, err_msg=<error
 ↪  reading variable: Cannot access memory at address
 ↪  0x9>, coeff=1, slope=0, f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
#14 star_solver::do_solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13, af1=...,
 ↪  ldaf=39, neq=76141,
 ↪  skip_global_corr_coeff_limit=.TRUE.,
 ↪  gold_tolerances_level=1,
    tol_max_correction=0.003000000000000001,
     ↪  tol_correction_norm=3.0000000000000001e-05,
     ↪  work=<error reading variable: value requires
     ↪  61521928 bytes, which is more than
     ↪  max-value-size>,
    lwork=7690241, iwork=<error reading variable: value
     ↪  requires 304564 bytes, which is more than
     ↪  max-value-size>, liwork=76141,
     ↪  convergence_failure=.FALSE., ierr=0)
    at ../private/star_solver.f90:317
#15 0x00000000007c5823 in star_solver::solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13,
 ↪  skip_global_corr_coeff_limit=.TRUE.,
 ↪  gold_tolerances_level=1,
    tol_max_correction=0.003000000000000001,
     ↪  tol_correction_norm=3.0000000000000001e-05,
     ↪  work=<error reading variable: value requires
     ↪  61521928 bytes, which is more than
     ↪  max-value-size>,
    lwork=7690241, iwork=<error reading variable: value
     ↪  requires 304564 bytes, which is more than
     ↪  max-value-size>, liwork=76141,
     ↪  convergence_failure=.FALSE., ierr=0)
    at ../private/star_solver.f90:104
```

I think the interesting calls are #8 and #13.

- Function call 8 sets the hydro_vars, which later give the error.
- Function call 13 adjusts the corrections after the previous newton iteration was unable to converge.

To continue debugging, I need to figure out *where* the wrong density values come from.

To do this, I use grep to look for all occurrences of density assignments:

```
> grep -R "s% *lnd.*=" star/private
star/private/predictive_mix.f90:            s% lnd(k) = lnd_save(k)
star/private/hydro_rsp2_support.f90:        s% lnd(k) = log(s% rho(k))
star/private/micro.f90:                     s% lnd(k) = logRho*ln10
star/private/star_utils.f90:                s% lnd_start(k) = -1d99
star/private/rsp_eval_eos_and_kap.f90:      s% lnd(k) = logRho*ln10
star/private/rsp_eval_eos_and_kap.f90:      s% lnd(k) = logRho*ln10
star/private/rsp_eval_eos_and_kap.f90:      s% lnd(kk) = logRho_result*ln10
star/private/conv_premix.f90:               s%lnd(kc_t:kc_b) = sd%lnd(kc_t:kc_b)
star/private/hydro_vars.f90:                s% lnd(k) = s% xh(i_lnd,k)
star/private/hydro_vars.f90:                s% lnd_start(k) = s% lnd(k)
star/private/solver_support.f90:            s% lnd(k) = x(i_lnd)
star/private/struct_burn_mix.f90:           !s% lnd_start(k) = s% lnd(k) set elsewhere
star/private/write_model.f90:               call write1(s% lnd(k),ierr); if (ierr /= 0) exit
```

Really, there are only a two lines here, namely:

```
star/private/hydro_vars.f90:                s% lnd(k) = s% xh(i_lnd,k)
star/private/solver_support.f90:            s% lnd(k) = x(i_lnd)
```

We can again run the debugger and set breakpoints at these lines:

```
(gdb) b hydro_vars.f90:299
Breakpoint 1 at 0x50dbb2: file ../private/hydro_vars.f90, line 299.
(gdb) b solver_support.f90:1337
Breakpoint 2 at 0x7f7194: file ../private/solver_support.f90, line 1337.
(gdb) condition 2 x(i_lnd) < -30
(gdb) run
```

Eventually, we reach the state:

```
                 solver_call_number           1
               gold tolerances level          1
   correction tolerances: norm, max    3.0000000000000001D-05    3.0000000000000001D-03
toll residual tolerances: norm, max    9.9999999999999994D-12    1.0000000000000001D-09
1  1  coeff 1.0000 avg resid  0.691E-05     max resid dv_dt     1  0.83380E+00 mix type xx000
                    avg corr  0.206E-03      max corr lnd  1986  0.53235E-02 mix type 00000
                    avg+max corr+resid
1  2  coeff 1.0000 avg resid  0.683E-05     max resid dv_dt     1  0.82426E+00 mix type xx000
                    avg corr  0.573E-04      max corr lnd     2  0.12612E-01 mix type x0000
                    avg+max corr+resid

Breakpoint 2, set_vars_for_solver::set1 (k=1, report=.FALSE., ierr=0)
             at ../private/solver_support.f90:1337
1337                        s% lnd(k) = x(i_lnd)
(gdb) p x(i_lnd)
$1 = -93.395479040704487
(gdb) p s% solver_dx(i_lnd,k)
$2 = -67.602505267054823
```

This is the first place where the density becomes very wrong. And in fact, $\ln\rho = -93.39 \approx \log\rho = -40.56$. We see that at this point, `p s% solver_dx(i_lnd,k)= -67.60...`, which is a really extreme correction.

*Why is this?*

To figure this out, we need to dive deeper into the code responsible for the corrections.

```
> grep -R "s%.*solver_dx.*=" star/private
star/private/star_solver.f90:     s% solver_dx(i,k) = dxsave(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = s% solver_dx(i,k) + s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = s% solver_dx(i,k) + coeff*s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = dxsave(i,k) + s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i,k) = dxsave(i,k) + coeff*s% x_scale(i,k)*soln(i,k)
star/private/star_solver.f90:     s% solver_dx(i_var,k+k_off) = save_dx(i_var,k+k_off) + delta_x
star/private/star_solver.f90:     s% solver_dx(i_var_sink,k+k_off) = save_dx(i_var_sink,k+k_off) - delta_x
star/private/star_solver.f90:     s% solver_dx(i_var,k+k_off) = save_dx(i_var,k+k_off)
star/private/star_solver.f90:     s% solver_dx(i_var_sink,k+k_off) = save_dx(i_var_sink,k+k_off)
star/private/alloc.f90:           s% solver_dx(1:nvar,1:nz) => s% solver_dx1(1:nvar*nz)
star/private/alloc.f90:           s% solver_dx(1:nvar,1:nz) => s% solver_dx1(1:nvar*nz)
star/private/struct_burn_mix.f90: s% solver_dx(j1,k) = s% xh(j1,k) - s% xh_start(j1,k)
star/private/struct_burn_mix.f90: s% solver_dx(j1,k) = s% xa_sub_xa_start(j2,k)
```

I think the easiest way to proceed is to actually *watch* `s% solver_dx(i,k)`. In particular, we want to watch $i = 1$ and $k = 1$, where $k$ is the zone, and $i = 1$ corresponds to the $\ln\rho$.

To set this up, we need to make sure we can access the right object, so we have to take some preparation steps. This is the full list of commands.

```
> OMP_NUM_THREADS=1 /usr/bin/gdb ./star
(gdb) b do_solver
(gdb) run
(gdb) c
Continuing.

Breakpoint 1.2, star_solver::do_solver (s=0xe7c0e0
 ↪  <__star_data_def_MOD_star_handles>, nvar=13,
 ↪  af1=<error reading variable: value requires 23821592
 ↪  bytes, which is more than max-value-size>,
    ldaf=39, neq=76141,
     ↪  skip_global_corr_coeff_limit=.TRUE.,
     ↪  gold_tolerances_level=1,
     ↪  tol_max_correction=0.003000000000000001,
     ↪  tol_correction_norm=3.0000000000000001e-05,
    work=<error reading variable: value requires 61521928
     ↪  bytes, which is more than max-value-size>,
     ↪  lwork=7690241,
    iwork=<error reading variable: value requires 304564
     ↪  bytes, which is more than max-value-size>,
     ↪  liwork=76141, convergence_failure=.TRUE., ierr=0)
     ↪  at ../private/star_solver.f90:109
109               subroutine do_solver( &
(gdb) delete 1
(gdb) set $star = s
(gdb) watch $star% solver_dx(1,1)
(gdb) b star_solver.f90:848
```

Okay! Let's look at the results:

```
Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0
New value = 0.012397313114644006
apply_coeff (just_use_dx=<optimized out>, coeff=1,
 ↪  soln=<error reading variable: value requires 609128
 ↪  bytes, which is more than max-value-size>,
    dxsave=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  nz=5857, nvar=<optimized out>) at
     ↪  ../private/star_solver.f90:996
996                               do i=1,nvar
(gdb) c
Continuing.

Breakpoint 3, adjust_correction (_err_msg=256,
 ↪  _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
 ↪  f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
848                         call do_equations(ierr)
(gdb) p iter
$1 = 1
(gdb) c
Continuing.
      1  1  coeff 1.0000 avg resid  0.691E-05     max
       ↪  resid dv_dt     1  0.83380E+00 mix type xx000
       ↪  avg corr  0.206E-03      max corr lnd  1986
       ↪  0.53235E-02 mix type 00000  avg+max corr+resid

Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0.012397313114644006
New value = 0.33770784939148324
apply_coeff (just_use_dx=<optimized out>, coeff=1,
 ↪  soln=<error reading variable: value requires 609128
 ↪  bytes, which is more than max-value-size>,
    dxsave=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  nz=5857, nvar=<optimized out>) at
     ↪  ../private/star_solver.f90:996
996                               do i=1,nvar
(gdb) c
Continuing.

Breakpoint 3, adjust_correction (_err_msg=256,
 ↪  _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
 ↪  f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
848                         call do_equations(ierr)
(gdb) p iter
$2 = 1
(gdb) c
Continuing.
      1  2  coeff 1.0000 avg resid  0.683E-05     max
       ↪  resid dv_dt     1  0.82426E+00 mix type xx000
       ↪  avg corr  0.573E-04      max corr lnd     2
       ↪  0.12612E-01 mix type x0000  avg+max corr+resid

Hardware watchpoint 2: $star% solver_dx(1,1)

Old value = 0.33770784939148324
New value = -67.602505267054823
apply_coeff (just_use_dx=<optimized out>, coeff=1,
 ↪  soln=<error reading variable: value requires 609128
 ↪  bytes, which is more than max-value-size>,
    dxsave=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  nz=5857, nvar=<optimized out>) at
     ↪  ../private/star_solver.f90:996
996                               do i=1,nvar
(gdb) c
Continuing.

Breakpoint 3, adjust_correction (_err_msg=256,
 ↪  _err_msg=256, ierr=0, err_msg=..., coeff=1, slope=0,
 ↪  f=0,
    grad_f=<error reading variable: value requires 609128
     ↪  bytes, which is more than max-value-size>,
     ↪  max_corr_coeff=1, min_corr_coeff_in=<optimized
     ↪  out>) at ../private/star_solver.f90:848
848                         call do_equations(ierr)
(gdb) p iter
$3 = 1
(gdb)
```

Clearly, we are getting a wrong `s% solver_dx(1,1)` from `apply_coeff`. It gets changed like this:

```
s% solver_dx(i,k) = s% solver_dx(i,k) + s%
 ↪  x_scale(i,k)*soln(i,k)
```

I inspected `s% x_scale` and `soln` throughout the Newton Iterations. The scale is consistent, while the solution at some point just becomes very large.

It also makes sense that we see the following in the terminal directly following the wrong density:

```
do_eos_for_cell: get_eos ierr          -1
do_eos returned ierr          -1
set_hydro_vars failed in call to set_micro_vars
                          set_hydro_vars returned ierr
                           ↪  -1
         eval_equations: set_solver_vars returned ierr
          ↪  -1
```

This is because directly following the call to `apply_coeff`, the routine `adjust_correction` calls `do_equations`, which calls `eval_equations > set_solver_vals > set_vars_for_solver > set_hydro_vars > set_micro_vars > set_eos > do_eos > foreach_cell > do_eos_for_cell > get_eos`.

This is the place where the first error occurs due to a density that is too small.

We probably want to look how we obtain the wrong `soln(1,1)`.

The scale is computed according to `subroutine set_xscale_info(s, nvar, ierr)` in `solver_support.f90`.

Here, for normal variables[5], the scale is determined according to

[5] not rotation or abundances

```
s% x_scale(i,k) = max(1, abs(s% xh_start(i,k)))
```

The values we find for the scale align well with this equation, so it must be the solution to the Newton-Raphson solver that is wrong.

This scheme is represented by repeatedly trying to solve (Paxton et al., 2011)

$$0 = \vec{F}(\vec{y}) = \vec{F}(\vec{y}_i + \delta\vec{y}_i) = \vec{F}(\vec{y}_i) + \left[\frac{\mathrm{d}\vec{F}}{\mathrm{d}\vec{y}}\right]_i \delta\vec{y}_i + \mathcal{O}(\delta\vec{y}_i^2.$$

Here, $y_i$ is a trial solution, $\vec{F}(\vec{y}_i)$ is the residual of the trial solution, $\delta\vec{y}_i$ is the correction to the trial solution and $[\mathrm{d}\vec{F}\,/\,\mathrm{d}\vec{y}]_i$ is the Jacobian matrix. The "solutions" from MESA are the $\delta\vec{y}_i$'s, which get scaled by the scale to obtain the changes to the star.

For a single iteration, MESA solves for $\delta\vec{y}$ in

$$J\delta\vec{y} = -\vec{F}(\vec{y}).$$

To make sure the Newton-Raphson solver is working correctly, I access the Jacobian, correction and the residual.

I then compute the LHS and the RHS of the equation above and test their agreement[6] .

[6] for the first zone

```
lhs = dblk_1.T @ delta_corr_1 + ublk_1.T @ delta_corr_2
rhs = -1 * equ
print(lhs - rhs)
```

```
[9.43255890e-18 1.06581410e-14 0.00000000e+00
 ↪  2.41035957e-15
 3.64629801e-63 1.21543267e-63 2.09119527e-85
  ↪  0.00000000e+00
 0.00000000e+00 0.00000000e+00 0.00000000e+00
  ↪  0.00000000e+00
 0.00000000e+00]
```

Nice. The Newton-Raphson solver works! the Problem must occur *before* the solver.

This also means the absurd correction of $-2.634$ is the *correct* value based on the residuals and the Jacobian.

Let's see how the corrections are constructed, based on the individual components of the Jacobian and the residuals.

From now on, I will only consider the four `hydro_vars`. This means I am *excluding* the abundances, which do not seem to impact the first zone anyway.

The Jacobian for the self interaction of the first zone is defined like this:

$$\mathbf{J}_{1\to1} = \begin{bmatrix} \frac{\partial y_\rho}{\partial \ln\rho} & \frac{\partial y_\rho}{\partial \ln T} & \frac{\partial y_\rho}{\partial \ln R} & \frac{\partial y_\rho}{\partial \ln L} \\ \frac{\partial y_T}{\partial \ln\rho} & \frac{\partial y_T}{\partial \ln T} & \frac{\partial y_T}{\partial \ln R} & \frac{\partial y_T}{\partial \ln L} \\ \frac{\partial y_R}{\partial \ln\rho} & \frac{\partial y_R}{\partial \ln T} & \frac{\partial y_R}{\partial \ln R} & \frac{\partial y_R}{\partial \ln L} \\ \frac{\partial y_L}{\partial \ln\rho} & \frac{\partial y_L}{\partial \ln T} & \frac{\partial y_L}{\partial \ln R} & \frac{\partial y_L}{\partial \ln L} \end{bmatrix}_{1\to1}, \delta\vec{y}_1 = \begin{bmatrix} \delta y_\rho \\ \delta y_T \\ \delta y_R \\ \delta y_L \end{bmatrix}_1, \vec{F}(\vec{y}_1) = \begin{bmatrix} F(y_\rho) \\ F(y_T) \\ F(y_R) \\ F(y_L) \end{bmatrix}_1$$

Now, importantly, this self interaction is *not the only* part that influences the first zone. MESA has couplings to the next and previous cell, where the complete matrix is split in a block diagonal form like this:

$$\mathbf{A} = \begin{array}{c} \\ \text{zone 1} \\ \text{zone 2} \\ \text{zone 3} \\ \text{zone 4} \\ \text{zone 5} \end{array} \begin{array}{c} \begin{array}{ccccc} \text{zone 1} & \text{zone 2} & \text{zone 3} & \text{zone 4} & \text{zone 5} \end{array} \\ \begin{bmatrix} D_1 & U_1 & 0 & & \\ L_2 & D_2 & U_2 & 0 & \\ 0 & L_3 & D_3 & U_3 & 0 \\ & 0 & L_4 & D_4 & U_4 \\ & & 0 & L_5 & D_5 \end{bmatrix} \end{array}.$$

In the case of zone 1, only $D_1$ and $U_1$ matter[7].

[7] For a more general zone $k$, we have $L_k$, $D_k$ and $U_k$.